TRACK B: MACHINE LEARNING AT SCALE • HDFS ➔ SPARK ➔ SPARK MLLIB

Team Video Call Speaking Script & Live Commands Guide

Distributed Urban Air Quality Analytics & CPCB AQI Forecasting Platform

Audience: Evaluators / Viva Committee

Team Division: 3 Equal Roles (HDFS, Spark, MLlib)

Total Time: 8 – 10 Minutes + Q&A

🟢 PERSON 1: Data Ingestion & Hadoop HDFS Lead — ⏱️ 0:00 – 3:00 (3 Minutes)

STEP 1: SAY THIS FIRST (Natural Opening):

> *"Hi everyone, our project is* **Distributed Air Quality Analytics and AQI Forecasting**. *We implemented* **Track B** *using Hadoop HDFS, Apache Spark, and Spark MLlib.*
>
> *Continuous air quality monitoring stations produce heavy, continuous streams of pollution telemetry. A single machine using Pandas will hit out-of-memory errors when processing years of sensor data with sliding averages. So, we built a distributed architecture starting with Hadoop HDFS."*

💻 STEP 2: RUN THIS COMMAND IN TERMINAL (SHOW RAW DATA IN HDFS):

```
/Users/yashsonawane/bigdata/hadoop/current/bin/hdfs dfs -ls -h /air_quality/raw/
```

Expected Output on Screen: `-rw-r--r-- 3 yashsonawane 47.3 M /air_quality/raw/raw_hourly_measurements.jsonl`

SAY THIS WHILE THE OUTPUT SHOWS ON SCREEN:

> *"First, we pulled data from OpenAQ API v3 for three major Indian metros: Delhi, Bengaluru, and Hyderabad. We collected* **96,000 raw hourly sensor readings** *covering six criteria pollutants—PM2.5, PM10, NO2, SO2, CO, and Ozone.*
>
> *As you can see live in HDFS on my screen, this raw dataset is* **47.3 MB**. *We stored it in HDFS with a* **128 MB block size** *and a* **3x replication factor**, *making it completely fault-tolerant and safe across the cluster."*

💻 STEP 3: RUN THIS COMMAND (SHOW PARTITIONED PARQUET LAKE):

```
/Users/yashsonawane/bigdata/hadoop/current/bin/hdfs dfs -ls -h /air_quality/processed/hourly_aqi_parquet/
```

Expected Output on Screen: Folders: `city=Bengaluru`, `city=Delhi NCR`, `city=Hyderabad`

SAY THIS TO EXPLAIN PARTITIONS:

> *"Here in* `/air_quality/processed/`, *you can see our cleaned dataset stored in* **Apache Parquet**. *It is partitioned by city: Delhi, Bengaluru, and Hyderabad. Partitioning allows Spark workers to read only the folders they need instead of scanning all 47 MB every time, making our queries significantly faster."*

🤝 **HAND-OFF LINE:** "Now, I will pass it to **[Person 2's Name]**, who will explain how we cleaned the data and calculated the official Indian AQI using Apache Spark."

🔵 PERSON 2: Spark ETL & India CPCB Regulatory Lead ⏱ 3:00 – 6:00 (3 Minutes)

STEP 1: SAY THIS FIRST (Explain Spark ETL & CPCB Standards):

> *"Thank you, [Person 1]. Once the data sat in HDFS, I used* **Apache Spark 4.2.0** *to clean it and calculate the official Indian Air Quality Index.*
>
> *A major flaw in naive implementations is simply averaging raw pollutant concentrations. That is scientifically invalid: 60 of PM2.5 is moderate pollution, but 60 of SO2 is completely safe.*
>
> *We strictly implemented India's official* **Central Pollution Control Board (CPCB)** *methodology:*
> *1. We harmonized all gases from ppb to mass concentrations (µg/m³).*
> *2. We calculated 24-hour moving averages for particulates and 8-hour averages for CO and Ozone.*
> *3. We enforced the strict* **16-hour rule***: a day is only valid if there are at least 16 hours of real sensor readings. We never replace missing numbers with zero.*
> *4. We calculated sub-indices using CPCB linear breakpoints, and the highest sub-index becomes the final composite AQI."*

💻 STEP 2: RUN THIS COMMAND IN TERMINAL (RUN SPARK SQL ANALYTICS JOB):

```
export JAVA_HOME="/opt/homebrew/Cellar/openjdk@17/17.0.20.1/libexec/openjdk.jdk/Contents/Home"
.venv/bin/python3 scripts/run_analytics.py
```

Expected Output on Screen: Printed table with City Stats (Delhi PM2.5: 121.9, Bengaluru: 28.8) and Diurnal peak charts generated.

SAY THIS WHILE THE TABLES PRINT ON SCREEN:

> *"I'm now running our Spark SQL analytics job across the cluster. Out of 18,521 total hours,* **12,792 hours met the strict CPCB validity rule** *in just 25 seconds.*
>
> *Look at the table printed on screen:*
> • **Delhi NCR** *recorded a mean PM2.5 of* **121.9 µg/m³***, and* **62.5% of all valid hours fell in the 'Severe' category***.*
> • **Bengaluru and Hyderabad** *had much cleaner air, averaging around* **28 µg/m³***.*
> • *Furthermore, our diurnal chart proved that pollution follows a bimodal cycle: peaking sharply in the morning rush hour around* **8:00 AM***, and peaking again late at night around* **10:00 PM** *when cold air traps vehicular emissions near the ground."*

🤝 **HAND-OFF LINE:** "Now, I will pass it to **[Person 3's Name]**, who will show our distributed forecasting models in Spark MLlib."

🟣 PERSON 3: Spark MLlib Machine Learning & Forecasting Lead ⏱ 6:00 – 9:00 (3 Minutes)

STEP 1: SAY THIS FIRST (Explain ML Strategy & No Data Leakage):

> *"Thank you, [Person 2]. I engineered the distributed machine learning pipeline using* **Spark MLlib** *to forecast pollution 24 hours in advance.*
>
> *To ensure our machine learning evaluation was genuinely sound and realistic, we did two critical things:*
> *1.* **Zero Data Leakage:** *We did NOT use random K-Fold splits, because in time series, random splits cheat by leaking future weather into the past. We used a strict* **chronological 80/20 block split***— training on the first 10,294 historical hours and testing on the final 2,570 future hours.*
> *2.* **Honest Benchmarking:** *We benchmarked against a* **Naive Persistence Baseline***, which simply assumes tomorrow's pollution equals today's (yielding an RMSE of 42.16 µg/m³)."*

💻 **STEP 2: RUN THIS COMMAND IN TERMINAL (TRAIN SPARK MLLIB MODELS):**

```
export JAVA_HOME="/opt/homebrew/Cellar/openjdk@17/17.0.20.1/libexec/openjdk.jdk/Contents/Home"
.venv/bin/python3 scripts/train_forecasting_models.py
```

Expected Output on Screen: Model Benchmark: Baseline (42.16) ➔ Linear Regression (37.86) ➔ Random Forest (35.17, +16.6% improvement).

SAY THIS WHILE THE MODELS TRAIN & PRINT:

> *"I just ran our distributed training pipeline. Spark's `VectorAssembler` combined past temporal lags from 1 hour ago up to 24 hours ago, along with temperature and humidity.*
>
> *Look at the benchmark results on screen:*
> *• Baseline RMSE was* **42.16 µg/m³***.*
> *• Spark MLlib Linear Regression brought error down to* **37.86 µg/m³***.*
> *• Our best model—***Spark MLlib Random Forest***—achieved an* **RMSE of 35.17 µg/m³***! That is a* **16.6% improvement over baseline** *with an R^2 of 0.306, training in just* **1.49 seconds** *across executor cores!*
>
> *We also trained a Random Forest Classifier to forecast the 6 CPCB health hazard categories (Good, Moderate, Severe, etc.), achieving* **70.21% test accuracy***."*

SAY THIS TO CONCLUDE:

> *"In summary, we built a complete big data system: raw data into* **HDFS***, distributed cleaning and CPCB calculation in* **Apache Spark***, and fast 24-hour forecasting with* **Spark MLlib***. All code, figures, and our 1.3 MB report are on our GitHub. We're now happy to answer your questions. Thank you!"*

📋 Quick Reference Cheatsheet for Team Members

Person	What You Show	Exact Command to Type	Key Numbers to Recite
Person 1 (HDFS Lead)	Raw JSON Telemetry in HDFS	`hdfs dfs -ls -h /air_quality/raw/`	**96,000 readings**, 47.3 MB, 128 MB blocks, 3x replication factor
Person 1 (HDFS Lead)	Partitioned Parquet Data Lake	`hdfs dfs -ls -h /air_quality/processed/hourly_aqi_parquet/`	Partitioned by `city`, 92% compression ratio, Snappy format
Person 2 (Spark ETL Lead)	Distributed Spark SQL Analytics	`.venv/bin/python3 scripts/run_analytics.py`	**12,792 valid CPCB records**, 16h rule, Delhi 62.5% Severe
Person 3 (MLlib Lead)	Spark MLlib Model Training	`.venv/bin/python3 scripts/train_forecasting_models.py`	Baseline 42.16 ➔ **Random Forest 35.17 (+16.6%)**, 1.49s train time, 70.2% accuracy
Any Member	Full End-to-End Master Pipeline	`./run_pipeline.sh`	Executes all 5 stages in sequence with exit code 0

🛡️ Top 5 Viva Defense Questions & Answers (Who Answers What)

Q1: "Why use Hadoop HDFS and Apache Spark instead of Pandas and Scikit-Learn?" (Answered by Person 1)

Answer: "Pandas loads the entire dataset into a single computer's RAM, which crashes with out-of-memory errors as sensor telemetry grows to millions of rows. HDFS distributes data blocks across DataNodes, and Spark executes transformations and ML tree splits in parallel across executor cores in just 1.5 seconds."

Q2: "Can't we just calculate AQI by taking the average of all pollutant concentrations?" (Answered by Person 2)

Answer: "No, that is scientifically invalid. Different pollutants have completely different toxicities—for instance, 60 µg/m³ of PM2.5 is Moderate pollution, while 60 µg/m³ of SO2 is completely clean. CPCB requires piecewise linear sub-indices for each pollutant, with the final AQI determined by the maximum operator: AQI = max(I_1, I_2, ... I_n)."

Q3: "Did you use K-Fold Cross-Validation for your machine learning models?" (Answered by Person 3)

Answer: "No, because in time-series forecasting, random shuffling leaks future weather into past training sets. We implemented a strict chronological 80/20 block split: the first 80% historical timestamps for training, and the final 20% strictly future horizon for testing. This guarantees zero data leakage."

Q4: "Why did you benchmark against a Naive Persistence Baseline?" (Answered by Person 3)

Answer: "Air pollution has strong autocorrelation: tomorrow's pollution often resembles today's. Reporting an R^2 or RMSE without a baseline is misleading. Our Random Forest model beat the persistence baseline by 16.6%, proving that our lag feature engineering learned real atmospheric dynamics."

Q5: "How did you handle missing values in sensor streams?" (Answered by Person 2)

Answer: "We never imputed missing values with zero, because zero implies clean mountain air and would falsely drag averages down. Instead, we enforced CPCB's statutory 16-hour completeness rule: if a 24-hour window had fewer than 16 valid readings, the AQI was marked invalid."